

Reducer-Owned Truth and Failure-Vaulted Retries: Host-Layer Primitives for LLM-Integrated Interactive Systems

Anthony Paterson

Fractal Media Infrastructure

Gracemere, Queensland, Australia

fractalmediainfrastructure@gmail.com

Author manuscript preprint, v0.1

Not peer reviewed.

Reducer-Owned Truth and Failure-Vaulted Retries: Host-Layer Primitives for LLM-Integrated Interactive Systems

Author details removed for review.

Abstract

Many safeguards for large language model systems are framed at the level of model alignment, benchmark performance, policy, or downstream human review. Those layers matter, but once a model is embedded in an interactive host, a practical trust question becomes architectural: who is allowed to change lasting state, what counts as evidence, how are failed attempts preserved, and what prevents fluent output from becoming authoritative fact? This paper proposes two small host-layer primitives for LLM-integrated systems: reducer-owned truth and failure-vaulted retry. Reducer-owned truth means canonical state is mutated only through deterministic reducer boundaries; model output may propose actions, narration, plans, summaries, or interpretations, but cannot directly become saved truth. Failure-vaulted retry means failed or stuck attempts are preserved as typed evidence, retried only with inspectable variation, compared across attempts, and promoted into future constraints only through explicit host-owned review. The paper develops these primitives through anonymized local-first reference implementations: a reducer-governed state core, a failure-aware routing core, a governed artifact-production pipeline, a source-of-truth planning system, a compact interactive mobile domain, and an adventure/game domain. These systems show how buckets, actions, reducer results, intent freezes, work-order gates, verification receipts, vault entries, retry deltas, triangulation records, and promoted constraints can make LLM-adjacent behavior more inspectable without granting the model hidden authority. The contribution is not a proof of safety and not a general solution to model unreliability. It is a design pattern for keeping authority, mutation, failure, and retry visible at the host layer.

CCS Concepts: Human-centered computing -- Human computer interaction (HCI); Human-centered computing -- Interactive systems and tools; Software and its engineering -- Software architectures; Computing methodologies -- Artificial intelligence.

Keywords: human-AI interaction; interactive intelligent systems; software architecture; auditability; large language models; failure handling

1. Introduction

The central risk in many LLM-integrated applications is not simply that a model may be wrong. It is that a model may be wrong while occupying a position where its output quietly becomes state. A generated summary may become the database. A conversational promise may become a hidden requirement. A failed tool call may be smoothed into optimistic prose. A retry may repeat the same faulty route until it appears to have succeeded. A model may be asked to define,

execute, and verify success inside the same loose interaction loop. In such systems, governance is not only a question of external review. It is a question of mutation boundaries.

This paper argues for two small primitives that can be added to model-integrated software without requiring a new model architecture. The first is reducer-owned truth. A reducer is a deterministic mutation boundary: it receives structured actions, checks legality against current state and domain rules, mutates canonical state when permitted, rejects impossible actions honestly when not permitted, and emits structured events explaining what changed. In this frame, "truth" is a software architecture term. It means the canonical saved state the host agrees to treat as authoritative for a workflow. It is not a claim of factual certainty, model knowledge, sentience, or mind-like cognition.

The second primitive is failure-vaulted retry. A failure vault treats failed or stuck attempts as evidence-bearing artifacts rather than disposable inconveniences. A bounded attempt may fail, stall, or be blocked. The host records the attempt, failure class, lock signals, retry parent, and subsequent variation. In the governed artifact-production pipelines, repeated attempts are compared to isolate the lock point; only then may an evidenced failure pattern become a candidate constraint, and only explicit promotion allows that constraint to affect future admissibility. Here too, "learning" is an operational host-state term. The system changes later routing because saved evidence and reviewed constraints have changed. This does not imply hidden model self-training, consciousness, or personhood.

The paper develops these primitives through six anonymized local project artifacts. A reducer-governed deterministic state core defines the minimal mutation surface. A failure-aware routing core defines a failure-to-constraint loop for bounded attempts, vault entries, retry deltas, triangulation sessions, constraint records, and routing decisions. A governed artifact-production pipeline applies these ideas to local agentic artifact production in which the user supplies intent and authority, the LLM supplies method proposals, and the host owns admissibility, execution bounds, evidence, constraints, and verification. A source-of-truth planning system supplies a related planning and snapshot discipline. A compact interactive mobile domain and an adventure/game domain show the same boundary in expressive settings where presentation, narration, media, and user interface state remain downstream of reducer-confirmed canonical state.

The contribution is modest by design. Rather than propose a total governance framework, this paper isolates a copyable architectural move: keep lasting truth boring, explicit, and reducer-owned; keep failure visible, typed, and usable; and keep model outputs in proposal or presentation lanes unless a host-owned process admits them into state.

2. Method and Evidence Boundary

This is an architecture/design paper, not a benchmark report, user study, or safety proof. Its claim is that reducer-owned truth and failure-vaulted retry are useful host-layer primitives for separating proposal, mutation, failure evidence, retry, and constraint promotion in LLM-

integrated interactive systems. The paper supports that claim by defining the primitives, mapping the authority boundaries they require, and grounding them in local implementation artifacts that already exercise reducer mutation, rejection, save/load preservation, vaulted failure, retry variation, constraint promotion, and verification against frozen intent.

The evidence is intentionally bounded. Existing source artifacts, tests, implementation charters, and worked traces are used as design evidence that the primitives can be represented concretely in software and can organize reviewable authority paths. The paper does not claim statistically measured reliability improvement, general safety, usability benefit, or benchmark superiority. Those claims would require additional empirical work. For the present contribution, the necessary evidence is narrower: clear definitions, one worked failure-to-constraint trace, and source-anchored examples showing that the relevant boundaries have executable or inspectable form.

A larger trace corpus, formal invariant test table, comparative reconstruction study, or user-facing evaluation would strengthen later versions. They are presented here as future validation rather than prerequisites for the narrower architectural claim.

3. Problem: Fluent Output as Hidden Authority

LLM systems are often evaluated by the quality of their visible outputs. In deployed applications, however, much of their power comes from invisible position. A model output may be treated as a plan, a memory, a fact, a command, a verification result, or a user preference depending on where the host places it. The same string can be harmless in a chat transcript and dangerous as a database mutation. The same failure can be useful as diagnostic evidence or harmful if concealed by fluent apology.

Three recurring problems motivate reducer-owned truth and failure-vaulted retry.

First, state can become scattered. Applications that grow around conversational agents often accumulate UI flags, chat history, improvised summaries, tool side effects, partial saves, and model-generated descriptions. When no one boundary owns canonical mutation, it becomes unclear what the system actually believes, what can be replayed, and which surface should be trusted after disagreement.

Second, prose can become an unofficial database. If a claim affects mechanics, permissions, obligations, safety posture, or future workflow, prose alone is too weak a container. Summaries are valuable as support context, but they should not be the sole store for mechanically or procedurally consequential claims. The same applies to model narration. A narrator can describe an outcome richly, but it should not be able to invent a location, grant an item, complete an objective, or authorize a build step merely because the prose sounds plausible.

Third, failure is often either hidden or repeated without learning. A failed attempt may produce a generic error, a superficial retry, or a model-generated explanation that is not connected to future behavior. A retry that does not differ materially from the failed attempt is not meaningful search.

A lesson that is not linked to evidence is not a trustworthy constraint. A successful later attempt should not erase the need to understand a previous failure if that failure may recur under related conditions.

These problems are familiar in software engineering under different names: single sources of truth, event logs, authority boundaries, state machines, audit trails, failure analysis, least privilege, and defense in depth. The novelty here is not that reducers or audit records are new. It is that LLM-integrated systems need these older principles reasserted at the point where model fluency tempts the host to collapse proposal, execution, verification, and memory into a single stream of text.

4. Primitive 1: Reducer-Owned Truth

Reducer-owned truth begins with a rule: if it changes lasting state, it must pass through a reducer. The reducer receives structured action input, evaluates legality, mutates canonical state if allowed, rejects the action if not allowed, and emits events and human-facing lines that describe the confirmed result.

In the reducer-governed state core used as a source artifact for this paper, canonical state is stored in buckets. Actions request changes. Reducers approve or reject those changes. Events explain what happened. Save envelopes persist canonical state directly. The engine surface is deliberately small: `EngineState`, `EngineAction`, `EngineEvent`, `EngineOutcome`, `Reducer`, `apply_action`, and `EngineSaveEnvelope`. This minimality matters because the pattern is intended to be copied into many host domains without imposing one vocabulary. Buckets might be renamed memory, store, workflow stages, scene entities, repository blockers, project notes, or classroom activities. The doctrine remains the same: lasting truth lives in explicit state and changes through a reducer.

This reducer boundary creates four separations.

The first separation is between canonical mutation and description. The reducer mutates state; reducer results describe what changed; presentation layers decorate confirmed outcomes. In the adventure/game-domain doctrine, this becomes the sentence: templates define canon, buckets track state, reducer mutates truth, narrator describes truth, UI displays truth. Descriptions are flavor. Fields are truth. If a fact affects mechanics later, it must be represented in structured state or reducer-visible data.

The second separation is between support memory and canonical state. Rolling summaries can help continuity, diagnostics, handoff, and narration, but they do not replace truth. In a reducer-owned system, summary lines follow reducer-confirmed events and lines after canonical truth has settled. They are support side effects. They may follow truth, but they may not create truth.

The third separation is between command resolution and authoritative execution. A natural-language parser or LLM command resolver may translate user text into a bounded command candidate. That candidate must still be validated before the reducer runs. Parser failures stop

before reducer and narrator handling. The narrator receives reducer-confirmed context after parsing and reduction; it does not override rejection, alter state, or execute actions directly.

The fourth separation is between media and mechanics. Media can reinforce current focus, support scenario tone, decorate outcomes, and provide future image, video, or audio hooks. It must not create mechanics, imply unconfirmed truth, unlock content, or complete objectives. This becomes especially important in model-rich systems because generated media and generated prose can both exert persuasive pressure. Reducer ownership keeps presentation expressive without making it authoritative.

5. Primitive 2: Failure-Vaulted Retry

Reducer-owned truth governs mutation. Failure-vaulted retry governs recovery. The core loop is:

1. make a bounded attempt;
2. detect truthful failure or stuckness;
3. vault the failure as evidence;
4. retry differently;
5. triangulate the shared lock point;
6. promote an evidenced constraint only through an explicit boundary;
7. route future attempts with better boundaries.

The failure-aware routing core used as a source artifact for this paper implements the lower-level mechanics as a small operational loop. Its state contains attempts, vault entries, constraints, triangulation sessions, and route logs. An attempt can include a method, summary, outcome, failure class, lock signals, and retry parent. A failed or stuck attempt creates or updates a vault entry. A retry delta records whether the method, summary, or lock signals changed. A triangulation session compares attempts within a failure class to isolate a shared lock point. A host can metabolize a vault entry into a constraint record containing the failure class, forbidden pattern, replacement guidance, and lock signals; routing can then block future attempts whose failure class or lock signals match that constraint. The governed artifact-production pipelines layer the stricter policy used in this paper's design pattern: comparative triangulation, candidate formation, and explicit promotion before a constraint affects future admissibility.

The important feature is not the terminology. Terms such as "entropy," "folding," and "metabolization" are local engineering metaphors. In the bounded operational sense used here, entropy means unresolved search disorder; folding means converting that disorder into usable structure; metabolization means promoting evidenced failure into a constraint. The mechanism is ordinary and inspectable: failure becomes evidence; evidence becomes triangulation; triangulation becomes constraint; constraint becomes future routing.

Failure-vaulted retry addresses four weaknesses in naive retry loops.

First, it rejects pretend success. A system should not blur a hard block into vague optimism. If it cannot complete honestly, cannot verify, or cannot continue without violating constraints or reality, it should say so and preserve why.

Second, it requires retry variation. A retry should differ in a bounded and inspectable way: smaller decomposition, different order, different method, different tool posture, tighter scope, or another evidence-justified change. Repeating the same failure path is not learning.

Third, it treats multiple failures as comparative evidence rather than isolated embarrassments. The purpose of repeated attempts is not brute force. It is triangulation: comparing failed or divergent attempts to identify the failure mechanism.

Fourth, it distinguishes candidate lessons from enforced constraints. Failure evidence may suggest a boundary, but a lesson should not become blocking doctrine merely because it was observed once. The host needs a promotion step that records why the constraint is scoped, what evidence supports it, and when it should apply.

6. Primitive Map

Primitive	Host responsibility	Model-permitted role	Not this
Canonical state	Store workflow facts that affect continuity, mechanics, obligations, or future routing	Describe, summarize, or ask to update	Chat transcript as the only database
Reducer	Approve or reject structured mutations against current state and rules	Propose candidate actions or explain results after confirmation	Model-authored state mutation
Event receipt	Record what changed or why an action rejected	Draft human-readable wording around confirmed events	Prose that substitutes for saved state
Bounded attempt	Define one clear move with scope, method, and expected evidence	Suggest a method or decomposition	Open-ended autonomous continuation
Failure vault	Preserve failed or stuck attempts as typed evidence	Help classify visible failure signals	Shame pile, deletion bin, or vague memory
Retry delta	Record what materially changed between attempts	Propose alternative routes	Repeating the same attempt with new wording
Triangulation	Compare attempts to isolate a shared lock point	Suggest hypotheses to test	Treating one failure as universal doctrine
Promoted constraint	In governed-pipeline policy, convert reviewed evidence into future routing boundaries	Recommend candidate constraints	Hidden model preference or unsupported habit
Verification receipt	Check outcome against frozen intent or reducer-confirmed rules	Suggest possible checks	Model defining its own success

This table is deliberately small. The primitives do not require all systems to use the same field names, schema, or storage layer. They require that consequential state, failure evidence, retry variation, and constraint promotion have explicit host-owned representations.

7. Auditable Criteria

The following criteria are the minimum review questions implied by the primitives. Some can be implemented as tests in particular systems; others are design-review questions. For the present architectural contribution, they are used as auditable criteria rather than as a claim that every supporting project already provides a complete invariant suite.

Invariant	Inspectable question	Example failure if violated
Reducer authority	Which reducer accepted this state change?	A narrator grants an item or completes a task without reducer confirmation
No mutation on rejection	Did canonical state remain unchanged after a rejected action?	Impossible command partially updates state
Proposal is not mutation	Can model output become saved state without a host boundary?	A summary silently overwrites project facts
Summary follows truth	Can summaries create mechanics-relevant facts?	Rolling memory adds a requirement never confirmed by the user
Failure is evidence	Is a failed attempt preserved with class, signals, and context?	A failed route disappears behind a vague apology
Retry is different	Does the retry delta show material variation?	The same bad route repeats until it seems normal
Constraints require promotion	Which evidence and review boundary produced this constraint?	One failure becomes a hidden permanent doctrine
Verification is external to proposal	Is success checked against frozen intent or reducer rules?	The model proposes a test and declares itself done
Human authority remains explicit	Which operator action confirmed intent or promoted consequence?	The host attributes unconfirmed derived labels to the user

These invariants do not prove a system safe. They make specific authority failures easier to detect. They also convert broad worries about "hallucination" into more concrete questions about where a string was allowed to travel.

8. Source-Anchored Implementation Evidence

The reducer-governed state core supplies the reducer-owned state primitive. Its terminology boundary is explicit: truth means reducer-governed saved state, not metaphysical truth or model-owned knowledge. This boundary keeps a useful engineering shorthand inside its intended architectural scope. The core also keeps the copyable surface small: state, action, event, outcome, reducer, apply function, and save envelope. Its tests show that reducer actions can update named

buckets, workflow-style reducers can change stage and blocker state, and save/load round trips can preserve state, templates, buckets, notes, and events.

The failure-aware routing core supplies the failure-vaulted retry primitive. It defines bounded attempts, stuckness, vaults, retry deltas, triangulation, lock points, metabolized constraints, and routing consequences. Its terminology boundary is likewise explicit: system learning means host-state adaptation through accumulated failure evidence and constraints, not hidden model training or personhood. Its tests show that failed attempts create vault entries, retry deltas record meaningful change, triangulation can produce a shared lock point, host-metabolized constraints can block related future attempts, and save/load round trips can preserve engine state.

The governed artifact-production pipeline applies both patterns to agentic artifact production. Its implementation charter splits authority into user, LLM, host, and output roles. The user supplies intent and authority. The LLM supplies externally provided positive method proposals. The host owns funneling, admissibility, execution bounds, evidence, constraints, and verification. It also supplies the stricter failure-vault policy: repeated convergent evidence produces a candidate, and an explicit promotion boundary is required before the candidate becomes an enforced constraint. Output is either a working artifact or truthful evidenced failure. The charter also rejects fallback substitution: a failed route does not license the system to silently change the product into the nearest supported shape. Its working-factory tests exercise intent confirmation, method proposal, verification receipts, provider failure stops, preserved prior failures, and resumed attempts that receive host-provided failure feedback.

A source-of-truth planning system contributes a related planning discipline. It treats plan files, project snapshots, and deterministic scans as source-of-truth anchors for the runner and planner. The relevance here is not that this system already has the same reducer/vault formalism as the two cores. It is that the ecosystem repeatedly distinguishes canonical artifacts from freeform generated prose.

The compact interactive mobile domain supplies a small setting where the distinction is visible. Its reducer specification states that impossible actions reject honestly, speech comes from reducer-confirmed outcomes, and UI selection is represented in structured state rather than hidden widget-only memory. Its tests show rejected inspections, item consumption through reducer results, event and line emission, unlocks through reducer-confirmed play/eat loops, stage clearing, selected-item cleanup, and clean rejection when no selectable item exists. Future AI narration may decorate confirmed events but cannot create item state, character state, unlocks, save data, or outcomes unless the reducer confirms them.

The adventure/game-domain doctrine supplies the same pattern in another expressive domain: templates define canon; buckets track state; reducer mutates truth; narrator describes truth; UI displays truth. Such domains are useful because authority leaks become obvious. If the narrator says the player has won while the objective state disagrees, the contradiction is visible. If an item appears in prose but not inventory, the authority failure is obvious. If media suggests a state change that the reducer did not confirm, the architecture has leaked.

9. Worked Trace: From Failed Attempt to Constraint

The following trace shows how the primitives fit together in a single host-mediated workflow.

1. The user asks for an artifact or state change.
2. The host preserves the exact request text, request bytes, request hash, and relevant source spans.
3. The operator confirms the intent freeze, including hard requirements and acceptance criteria.
4. The model proposes a method for satisfying the frozen intent.
5. The host creates a bounded work order only if the proposal is admissible.
6. The runner executes within the host's capability and scope bounds.
7. The verifier checks the outcome against frozen intent, not against model-convenient success criteria.
8. If the attempt succeeds, the host records artifact and verification receipts.
9. If the attempt fails or gets stuck, the host creates a vault entry with failure class, lock signals, evidence, and attempt context.
10. A retry is permitted only with a recorded delta showing what changed.
11. After repeated related failures, the host triangulates the shared lock point.
12. A scoped constraint candidate is produced from the evidence.
13. A human or explicit host-owned promotion boundary decides whether the candidate becomes an enforced future constraint.
14. Future routing consults the promoted constraint and blocks known bad routes under defined conditions.

The trace is mundane by intention. It gives a reviewer concrete artifacts to ask for: request hash, intent freeze, method proposal, gate receipt, work order, execution evidence, verification receipt, vault entry, retry delta, triangulation record, constraint candidate, and promotion record.

10. Future Validation Agenda

The projects discussed here are not presented as a completed empirical evaluation. That is a limitation on what the paper may claim, not a defect that prevents the narrower architecture/design contribution. The following layers describe useful future validation if later versions wish to claim reliability gains, usability gains, or superiority over transcript-only systems.

The first layer is broader invariant and property testing. For reducer-owned truth, later tests could assert that rejected actions leave canonical state unchanged, all mechanics-relevant facts live in structured fields, save/load preserves state, summaries cannot create state, narrator outputs cannot mutate state, and media focus follows reducer-confirmed events. For failure-vaulted retry,

later tests could assert that failed attempts produce vault entries, retry deltas record material change, constraints require evidenced promotion, and metabolized constraints block related future attempts while permitting unrelated ones. Existing tests already support parts of this layer, but a complete cross-project invariant suite would be a useful extension rather than a necessary condition for the present claim.

The second layer is comparative workflow evaluation. A study could compare LLM-direct mutation against reducer-mediated mutation across tasks such as game command resolution, project memory updates, code artifact creation, or workflow stage management. Outcome measures could include invalid state transitions, unverifiable claims, repeated identical failures, silent task substitution, recovery time after failure, reviewer effort, and user trust calibration. Such a study would be necessary only if the paper claimed measured comparative advantage.

The third layer is audit reconstruction. Independent reviewers could be given final artifacts and logs from several sessions and asked to reconstruct what happened, what failed, what changed, and why a future constraint exists. The hypothesis is that receipt-bearing reducer and vault architectures reduce reconstruction ambiguity compared with transcript-only or prose-summary systems.

The fourth layer is adversarial authority testing. In artifact-production systems, tests can attempt to fabricate receipt IDs, inject promoted constraints, supply host bounds, authorize work orders from gate receipts alone, replay stale authority, or bypass verification. Compile-fail and runtime tests can then demonstrate which authority paths are unavailable by construction and which still require runtime checks.

The fifth layer is user-facing failure evaluation. Participants could compare systems that hide, smooth, or repeatedly retry failures against systems that preserve failure evidence and explain changed attempts. Measures should avoid asking only whether users prefer smoothness. The relevant questions are whether users can tell what happened, whether they can identify unresolved risk, and whether they can decide when to stop, revise scope, or authorize a new attempt.

11. Related Work and Positioning

This work sits at the intersection of human-AI interaction, software architecture, safety engineering, and responsible technology. Human-AI interaction guidelines emphasize intelligibility, controllability, graceful failure, and appropriate user reliance. Reducer-owned truth and failure-vaulted retry translate those concerns into host-layer design questions: what boundary admits a claim into state, what receipt explains a change, and what artifact lets a reviewer reconstruct failure?

Software architecture offers related ideas in state machines, event sourcing, audit trails, least privilege, and deterministic replay. The paper does not claim novelty for reducers, logs, or

receipts in isolation. Its contribution is to position these old virtues against a specific LLM-era failure mode: the collapse of proposal, mutation, verification, and memory into fluent text.

Responsible AI work emphasizes governance, accountability, documentation, and risk management. These are often treated at organizational or model-evaluation levels. The primitives proposed here operate closer to the runtime host. They do not replace policy, review, privacy controls, or evaluation. They make those processes easier to inspect by producing host-owned evidence about how claims, failures, and constraints entered the system.

12. Limitations and Non-Claims

This paper does not claim that reducers make LLM systems safe. Reducers can encode the wrong rules, omit important state, or reject useful behavior. A badly designed reducer can centralize error rather than prevent it. The point is not automatic correctness, but inspectable authority.

The paper does not claim that failure vaults eliminate uncertainty. A vault may contain misclassified failures, incomplete evidence, or overfitted lock signals. Constraint promotion can be too conservative or too permissive. The governance value comes from preserving the path from evidence to constraint, not from assuming all promoted constraints are correct.

The paper does not claim that models learn internally from vaulted failures. The learning described here is host-state learning: saved evidence and promoted constraints affect future routing. This distinction is important both technically and ethically. It avoids anthropomorphic claims while still recognizing that a system can improve operational behavior across attempts.

The paper does not claim that all consequential facts are easy to structure. Some domains require interpretation, negotiation, judgment, and human review. Reducer-owned truth should not become a false demand that all meaning be flattened into fields. The narrower claim is that once a fact has operational consequences, the host should provide an accountable path by which it becomes state.

The paper does not replace external governance. Model evaluations, security review, privacy review, accessibility review, legal compliance, organizational accountability, and human oversight remain necessary. Reducer-owned truth and failure-vaulted retry are host-layer primitives that make those larger processes easier to inspect.

13. Conclusion

LLM-integrated systems do not need to grant models hidden authority in order to use them well. They can place models in proposal, narration, summary, and interpretation roles while preserving canonical mutation for deterministic reducers and host-owned gates. They can treat failure as evidence rather than embarrassment, retry only with inspectable variation, triangulate recurring lock points, and promote constraints only through accountable receipt chains.

Reducer-owned truth and failure-vaulted retry are small primitives. Their smallness is the point. They are copyable, testable, and compatible with many host domains. They reintroduce old software virtues at the place where LLM fluency most easily erodes them: the boundary between saying something and making it true.

References

- ACM. (2026). *ACM Transactions on Interactive Intelligent Systems author guidelines*. Association for Computing Machinery.
- ACM. (2026). *Open access at ACM*. Association for Computing Machinery.
- Amershi, S., Weld, D., Vorvoreanu, M., Fournery, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., and Horvitz, E. (2019). Guidelines for human-AI interaction. *Proceedings of CHI 2019*. <https://doi.org/10.1145/3290605.3300233>
- Fowler, M. (2005). Event sourcing.
- Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3), 231-274. [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>
- Leveson, N. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*.
- Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., and Barnes, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *Proceedings of FAccT 2020*. <https://doi.org/10.1145/3351095.3372873>
- Saltzer, J. H., and Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308. <https://doi.org/10.1109/PROC.1975.9939>
- Suchman, L. (1987). *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press.
- Wieringa, R. J. (2014). *Design Science Methodology for Information Systems and Software Engineering*. Springer.

Declaration of Generative AI and AI-assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the author used OpenAI ChatGPT and OpenAI Codex to assist with drafting, structuring, literature targeting, and document preparation. After using these

tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the submitted work.

Declarations

Funding: No external funding was received for this work.

Competing interests: The author declares no financial competing interests. The author developed and maintains the implementation repositories used as source-artifact evidence. This relationship should be disclosed in the non-anonymous submission materials because the manuscript analyzes tools and documentation from that ecosystem.

Data and code availability: implementation repositories are currently anonymized for review.

Public repository links, tagged snapshots, archival snapshots, or anonymized source extracts can be supplied where venue policy permits.

Ethics review: the present architecture/design paper does not report human-subjects research.

Later user studies or comparative participant evaluations would require ethics review according to venue and institutional requirements.